

->JOINS

📌 1. Title

Rising Temperature

🎯 Problem

- Find the **IDs of days** where the temperature was **higher than the previous day**.
- Only compare **consecutive days** (yesterday vs today).
- Return just the **id** of these days.

📄 Table

Weather

Column	Type	Description
id	int	Unique row identifier
recordDate	date	Date of the temperature record
temperature	int	Temperature recorded on that date

💡 Hint

- Use a **self join** to compare each day with the previous day.
- Find rows where `current_day.temperature > previous_day.temperature`.
- You can also use **window functions** (LAG)

✅ Solution

Option 1: Self Join

```
SELECT w1.id
FROM Weather w1
JOIN Weather w2
  ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

Option 2: Window Function

```
SELECT id
FROM (
  SELECT id,
         temperature,
         LAG(temperature) OVER (ORDER BY recordDate) AS prev_temp
  FROM Weather
) t
WHERE temperature > prev_temp;
```

2. Title

Employee Bonus

Problem

- Report the **name and bonus amount** of each employee who satisfies **either**:
 - Employee **did not get any bonus**.
 - Employee **bonus is less than 1000**.
- Return the result table in **any order**.

Tables

Employee

Column	Type	Description
empId	int	Unique ID of employee
name	varchar	Name of employee
supervisor	int	ID of supervisor (nullable)
salary	int	Employee salary

Bonus

Column	Type	Description
empId	int	Employee ID (FK to Employee.empId)
bonus	int	Bonus amount

Hint

- Use a **LEFT JOIN** to include employees without a bonus.
- Filter with: `bonus < 1000 OR bonus IS NULL`.

Solution

```
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b
  ON e.empId = b.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```